JWT Hacking

Introduction

JSON Web Tokens (JWTs) are compact and self-contained mechanisms for securely transmitting information between parties as a JSON object.

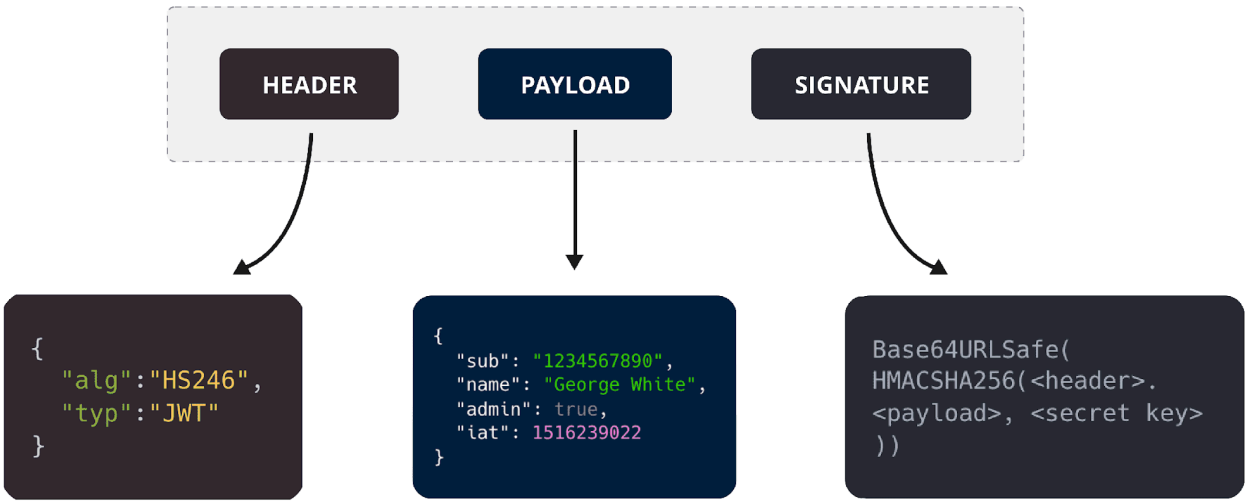
Features of JWT

- JWTs consist of three parts:
    - Header
    - Payload
    - Signature

They are encoded together to create a digitally signed token that can be easily verified and trusted.

- They support encryption and are signed using a private secret or a public/private key.

- They are URL safe and they can also be set as a cookie/session token.



Attack Vectors

1. **Leakage of Sensitive Information**

If the token is not encrypted and only encoded in base64 then it can be decoded easily to obtain sensitive information. To avoid this encryption needs to be performed which will provide both data integrity and confidentiality.

2. **None Algorithm Attack**

If the algorithm type is set to “none”, any token would be considered valid.

For example: { “alg”: “none”, … } - Will be considered valid.

3. **Kid Parameter Attack**

Kid is a special parameter in jwt metadata that is used to retrieve a key file from the filesystem. The parameter takes a file path as an attribute. If this path is not sanitized then it can enable several attacks such as directory traversal attack.

```
{
  “kid”: “/var/lib/file”
}
```

Here, the kid attribute can be modified to something like “../../../../../../etc/shadow” which is dangerous.

Impact

1. **Sensitive Information Disclosure**
   Standard JWTs are Base64 encoded rather than encrypted, any data in the payload is visible to anyone who intercepts the token.
2. **Authentication Bypass & Account Takeover**
   If signature validation is flawed (“none” algorithm attack), attackers can forge tokens to impersonate any user, including administrators.
3. **Unauthorized System Access**
   Manipulating header parameters like **`kid`** can lead to Directory Traversal or SSRF, allowing attackers to interact with the server's internal file system.
4. **Data Integrity Loss**
   An unverified token allows attackers to modify claims such as changing a **`user_id`** or **`role`** to gain privileges they were never granted.
5. **Database Exploitation**
   If decoded JWT claims are passed directly into backend queries without sanitization, it can serve as an entry point for SQL injection or NoSQL injection attacks.

Practical (Lab)

Completed lab at PortsWigger
https://portswigger.net/web-security/jwt/lab-jwt-authentication-bypass-via-unverified-signature

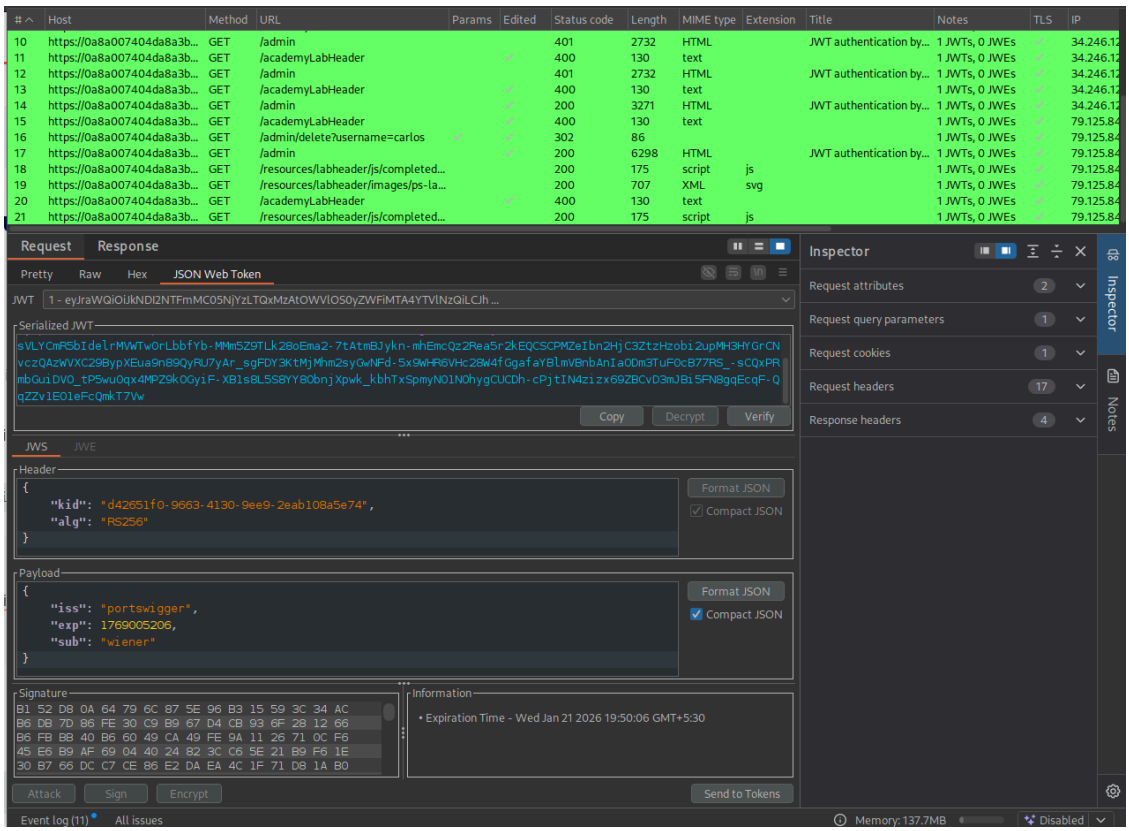
Lab: JWT authentication bypass via unverified signature

APPRENTICE
LAB Solved

Practical (Tool)

Learned to use JWT Editor extension in BurpSuite.

This extension provides tools to easily decode, encode & sign JWT tokens.



Conclusion

The security of JWT-based authentication relies entirely on the integrity of the signature and the strictness of the server-side validation. As demonstrated, simply encoding data in a token is not security; without robust signature verification, a JWT is merely a transparent data structure vulnerable to manipulation.